

1. Introducción y Herramientas

- El desarrollo moderno requiere dominar herramientas como **VSCode**, un editor extensible que utiliza extensiones como ESLint para errores, Prettier para formato y Live Server para recarga automática.
- Las **DevTools** del navegador (F12) son esenciales para depurar; el panel *Sources* permite inspeccionar archivos y *Console* ejecutar comandos.
- El uso de **puntos de interrupción (breakpoints)** es una técnica de depuración superior a `console.log()` para examinar el estado de variables en tiempo real.
- Las pestañas **Watch** y **Scope** en las DevTools permiten supervisar expresiones específicas y ver variables definidas en el contexto actual.

2. Fundamentos de JavaScript

- Las variables han evolucionado de `var` (ámbito global/función) a `let` y `const` (ámbito de bloque), recomendándose `const` para valores que no se reasignan.
- Maneja tipos de datos comunes como **String**, **Number**, **Boolean**, **Array** y **Object**, y utiliza el triple igual (`===`) para comparaciones estrictas de valor y tipo.
- Las funciones pueden ser **declarativas** (con *hoisting*), **expresadas** o **de flecha**, siendo estas últimas ideales para *callbacks*.
- Las **estructuras de control** (`if . . . else`, `switch`) y bucles (`for`, `while`) permiten modificar la ejecución lineal del programa.
- El bucle moderno `for . . . of` facilita la iteración sobre elementos de objetos iterables como arrays de forma legible.

3. Manipulación del DOM

- El **DOM** representa la página como un árbol de nodos, se seleccionan elementos mediante `getElementById`, `querySelector` o `querySelectorAll`.
- Se pueden modificar contenidos con `textContent`, estilos con la propiedad `style`, atributos con `setAttribute` y clases mediante `classList`.
- Los **eventos** (click, input, submit) se gestionan con `addEventListener`, en formularios es vital usar `event.preventDefault()` para evitar recargas.
- La **delegación de eventos** optimiza el rendimiento al asignar un único escucha al padre y detectar el origen mediante `event.target`.

4. JSON: Estructura y Manipulación

- **JSON** es el estándar ligero para intercambio de datos, exige que las claves usen comillas dobles y no admite funciones ni `undefined`.
- El método `JSON.stringify()` convierte objetos de JavaScript a cadenas JSON, algo indispensable para el uso de `localStorage`.
- El método `JSON.parse()` realiza el proceso inverso, transformando cadenas JSON en objetos o arrays manipulables.

5. Programación Asíncrona

- Debido a que JavaScript es de un solo hilo, utiliza la **asincronía** para no bloquear la interfaz durante tareas largas como peticiones a servidores.
- Las **Promesas** representan resultados eventuales (éxito o fracaso) con estados: *pending*, *fulfilled* y *rejected*.
- La sintaxis `async/await` permite escribir código asíncrono con una estructura que se lee como síncrona.
- Para el manejo de errores en flujos asíncronos, se deben envolver las llamadas en bloques `try...catch`.

6. AJAX y Fetch API

- **AJAX** permite actualizar partes de una web sin recargarla, la **Fetch API** es la interfaz moderna para realizar estas peticiones HTTP.
- Una petición `fetch` típica implica realizar la llamada a una URL y procesar la respuesta mediante el método `.json()`.
- Las **APIs REST** exponen recursos en URLs únicas, al consumirlas es común iterar resultados con `map` o `forEach`.
- Se recomienda el uso de **Templates HTML y Fragments** para inyectar datos en el DOM eficientemente, evitando el reflujo innecesario (*reflow*).